OBSERVER PATTERN - DETAILED ANALYSIS & IMPLEMENTATION

Flight Reservation System - Flight Price Change Notification

---

1. PATTERN IDENTIFICATION & LOCATION

Most Evident Location:

Class: `ScheduledFlight` (`src/main/java/flight/reservation/flight/ScheduledFlight.java` - Line 82)
Method: `setCurrentPrice(double currentPrice)` at line 82

Current Implementation (BEFORE Observer Pattern):

```
public void setCurrentPrice(double currentPrice) {
    this.currentPrice = currentPrice;
}
```

Problem: Price changes happen silently with no notifications to interested parties.

---

2. DETAILED PATTERN DESCRIPTION

What is the Observer Pattern?

The **Observer Pattern** is a behavioral design pattern that defines a **one-to-many dependency** between objects. When the state of one object (the **Subject**) changes, all its dependent objects (the **Observers**) are automatically notified and updated.

Key Components:

1. **Subject (Observable)**: The object being watched (e.g., `ScheduledFlight`)
2. **Observer**: Objects that want to be notified of changes (e.g., `Customer`, `Admin`, `PriceMonitor`)
3. **Concrete Subject**: Implements subject interface and maintains state
4. **Concrete Observers**: Implement observer interface and react to state changes

How It Works:

1. Observers **register** themselves with a Subject
2. Subject maintains a list of all registered Observers
3. When Subject's state changes, it **notifies** all Observers
4. Each Observer receives the notification and responds accordingly

---

3. APPLICATION TO THIS CODEBASE

Scenario: Flight Price Monitoring System

In airline booking systems, customers want to:

- **Track price changes** on flights they're interested in
- **Receive alerts** when prices drop below a threshold
- **Get notifications** when seats become available

Currently, when `setCurrentPrice()` is called on a `ScheduledFlight`, the price changes but **no one is notified**. This is a perfect use case for the Observer Pattern.

Subject (Observable):

- `ScheduledFlight` - The flight whose price changes

Observers (Who needs to know?):

1. **Customers** - Want price drop alerts for booking decisions
2. **Admin/Analytics Systems** - Track pricing trends and revenue
3. **Price Monitoring Services** - Aggregate price data across flights
4. **Notification Services** - Send emails/SMS to watchlist subscribers

---

4. REASONING & JUSTIFICATION

Why Observer Pattern is Ideal Here:

Natural One-to-Many Relationship

- ONE flight can have MANY customers watching it
- ONE price change needs to notify ALL interested parties

Loose Coupling

- `ScheduledFlight` doesn't need to know about `Customer`, `Admin`, or any specific observer type
- Can add new observer types (SMS service, analytics) without modifying `ScheduledFlight`

Open/Closed Principle

- System is **OPEN** for extension (add new observers)
- System is **CLOSED** for modification (don't change ScheduledFlight)

Real-World Business Value

- Airlines use dynamic pricing - prices change frequently
- Customers expect price alerts (Kayak, Google Flights do this)
- Revenue tracking requires price change monitoring

Separation of Concerns

- Flight management logic stays in `ScheduledFlight`
- Notification logic moves to Observer implementations
- Each class has a single responsibility

---

5. BENEFITS vs DRAWBACKS

BENEFITS:

Benefit	Explanation
Decoupling	Subject and Observers are loosely coupled. `ScheduledFlight` knows only about the `Observer` interface, not concrete implementations.
Scalability	Easily add/remove observers at runtime without modifying existing code.
Reusability	Observers can be reused across different subjects (e.g., same `CustomerNotification` for flights, hotels, car rentals).
Maintainability	Changes to notification logic don't affect flight management logic.
Testability	Can test `ScheduledFlight` and observers independently. Mock observers for testing.
Dynamic Subscription	Customers can subscribe/unsubscribe from price alerts dynamically.
Single Responsibility	Each class has one job: `ScheduledFlight` manages flights, observers handle notifications.

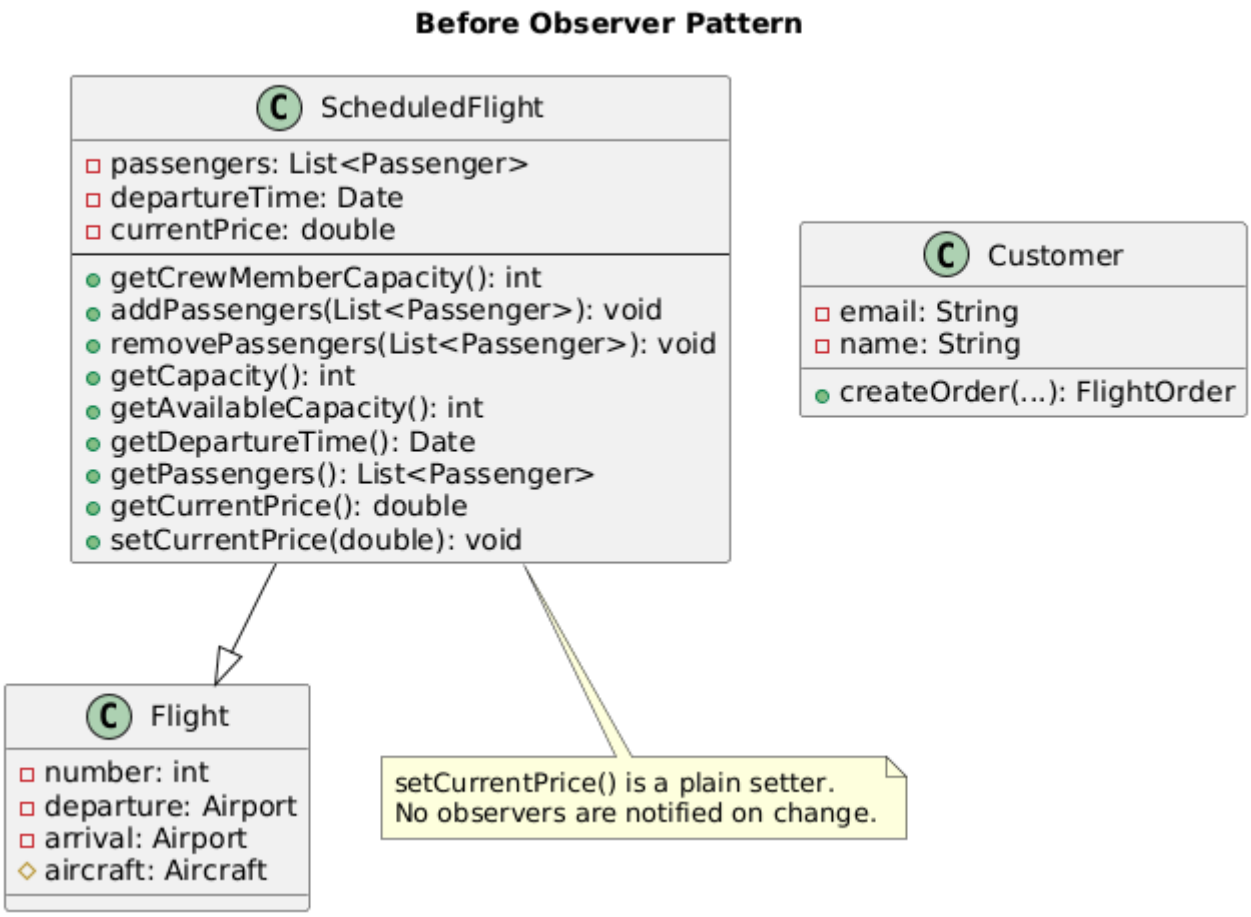
DRAWBACKS:

Drawback	Mitigation
Memory Overhead	Need to maintain observer lists.
Notification Order	Observers notified in arbitrary order.
Unexpected Updates	Observers might trigger cascading updates.
Notification Performance	Many observers = slower notifications.
Observer Leaks	Forgotten observers stay in memory.
Debugging Complexity	Hard to trace who caused what update.

---

6. CLASS DIAGRAMS

6.1 BEFORE OBSERVER PATTERN

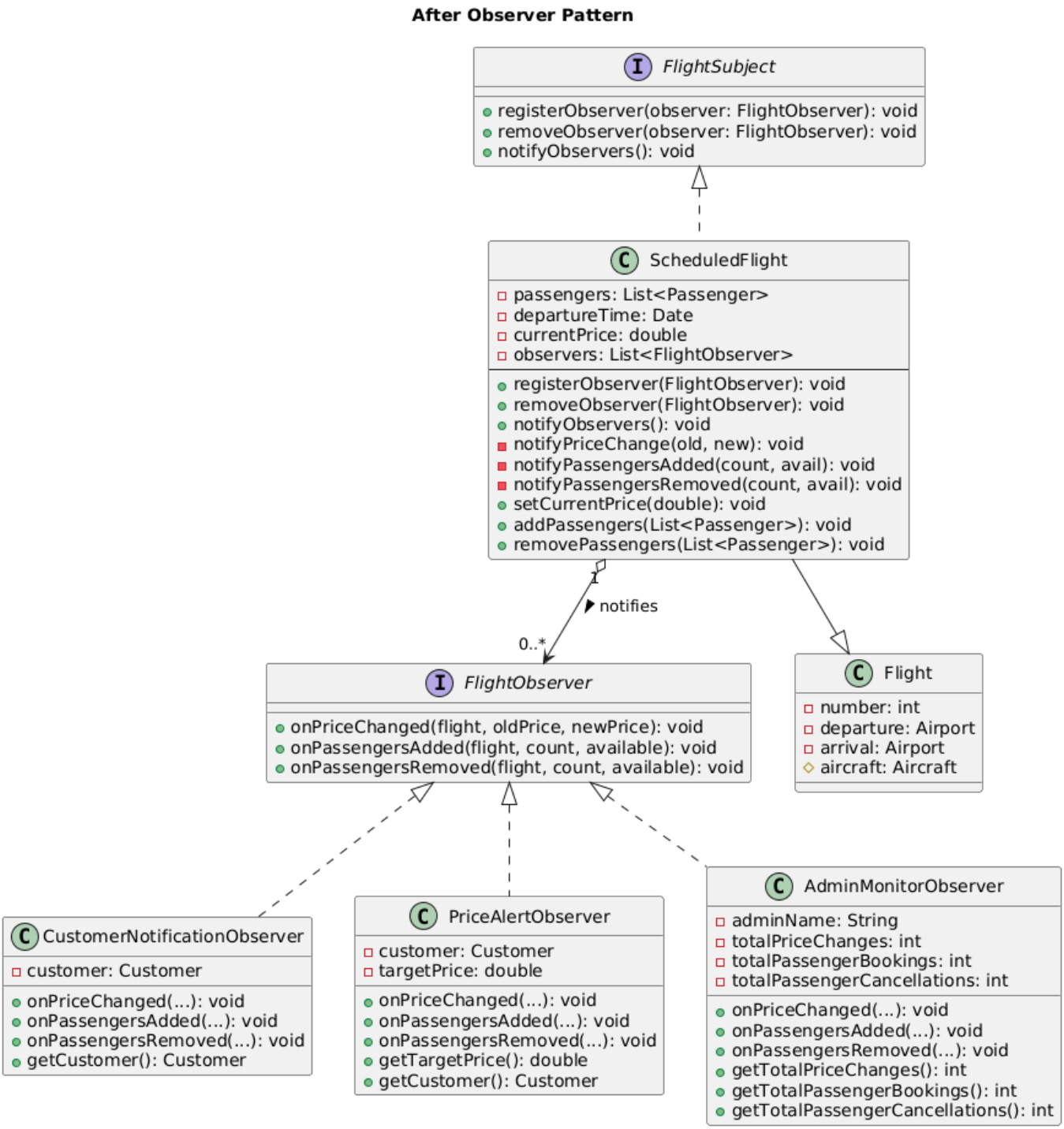


Analysis:

- `ScheduledFlight.setCurrentPrice()` is a simple setter
- No mechanism to notify interested parties
- `Customer` has no way to track price changes
- To monitor prices, interested parties would need to poll repeatedly (inefficient)
- Adding notification logic directly in `setCurrentPrice()` would tightly couple the class to specific notification types

---

6.2 AFTER OBSERVER PATTERN



Key Changes:

1. **New Interfaces**: `FlightObserver` and `FlightSubject`
2. **ScheduledFlight** now maintains `List<FlightObserver> observers`
3. **Multiple Observer Types** can register for notifications
4. **Loose Coupling**: ScheduledFlight only knows about the `FlightObserver` interface
5. **Open for Extension**: Can add new observer types without changing existing code

---

7. CODE IMPLEMENTATION

7.1 Observer Interface

```java
package flight.reservation.flight;

/** Observer interface — implement to receive ScheduledFlight state-change events.
*/
public interface FlightObserver {

    /** Called when the flight price changes. */
    void onPriceChanged(ScheduledFlight flight, double oldPrice, double newPrice);

    /** Called when passengers are added. */
    void onPassengersAdded(ScheduledFlight flight, int passengersAdded, int
availableSeats);

    /** Called when passengers are removed. */
    void onPassengersRemoved(ScheduledFlight flight, int passengersRemoved, int
availableSeats);
}
```

7.2 Subject Interface

```java
package flight.reservation.flight;

/** Subject interface — defines observer registration contract for
ScheduledFlight. */
public interface FlightSubject {

    /** Register an observer; must not be null. */
    void registerObserver(FlightObserver observer);

    /** Unregister a previously registered observer. */
    void removeObserver(FlightObserver observer);

    /** Notify all registered observers of a state change. */
    void notifyObservers();
}
```

7.3 Modified ScheduledFlight (Concrete Subject)

```java
package flight.reservation.flight;

import flight.reservation.Airport;
import flight.reservation.Passenger;
import flight.reservation.plane.Aircraft;

import java.util.ArrayList;
import java.util.Date;
```

```java
import java.util.List;

/** Concrete Subject — notifies registered FlightObservers on price/passenger
changes. */
public class ScheduledFlight extends Flight implements FlightSubject {

    private final List<Passenger> passengers;
    private final Date departureTime;
    private double currentPrice = 100;

    private final List<FlightObserver> observers;

    public ScheduledFlight(int number, Airport departure, Airport arrival,
Aircraft aircraft, Date departureTime) {
        super(number, departure, arrival, aircraft);
        this.departureTime = departureTime;
        this.passengers = new ArrayList<>();
        this.observers = new ArrayList<>();
    }

    public ScheduledFlight(int number, Airport departure, Airport arrival,
Aircraft aircraft, Date departureTime, double currentPrice) {
        super(number, departure, arrival, aircraft);
        this.departureTime = departureTime;
        this.passengers = new ArrayList<>();
        this.currentPrice = currentPrice;
        this.observers = new ArrayList<>();
    }

    // ---------------------------------------------------------------------
    // Observer Pattern — FlightSubject implementation
    // ---------------------------------------------------------------------

    @Override
    public void registerObserver(FlightObserver observer) {
        if (observer != null && !observers.contains(observer)) {
            observers.add(observer);
            System.out.println("[Observer] Observer registered for Flight " +
this.getNumber());
        }
    }

    @Override
    public void removeObserver(FlightObserver observer) {
        if (observers.remove(observer)) {
            System.out.println("[Observer] Observer removed from Flight " +
this.getNumber());
        }
    }

    @Override
    public void notifyObservers() {
        // specialised notify helpers are used instead
    }
```

```java
    // ---------------------------------------------------------------------------
    // Observer Pattern — private notification helpers
    // ---------------------------------------------------------------------------

    private void notifyPriceChange(double oldPrice, double newPrice) {
        // snapshot for safe concurrent modification
        for (FlightObserver observer : new ArrayList<>(observers)) {
            observer.onPriceChanged(this, oldPrice, newPrice);
        }
    }

    private void notifyPassengersAdded(int count, int availableSeats) {
        for (FlightObserver observer : new ArrayList<>(observers)) {
            observer.onPassengersAdded(this, count, availableSeats);
        }
    }

    private void notifyPassengersRemoved(int count, int availableSeats) {
        for (FlightObserver observer : new ArrayList<>(observers)) {
            observer.onPassengersRemoved(this, count, availableSeats);
        }
    }

    // ---------------------------------------------------------------------------
    // Core flight methods (existing behaviour preserved; notifications added)
    // ---------------------------------------------------------------------------

    public int getCrewMemberCapacity() throws NoSuchFieldException {
        return this.aircraft.getCrewCapacity();
    }

    public void addPassengers(List<Passenger> passengers) {
        this.passengers.addAll(passengers);
        int availableSeats;
        try {
            availableSeats = getAvailableCapacity();
        } catch (NoSuchFieldException e) {
            availableSeats = -1; // unknown; observers receive -1 as sentinel
        }
        notifyPassengersAdded(passengers.size(), availableSeats);
    }

    public void removePassengers(List<Passenger> passengers) {
        this.passengers.removeAll(passengers);
        int availableSeats;
        try {
            availableSeats = getAvailableCapacity();
        } catch (NoSuchFieldException e) {
            availableSeats = -1; // unknown; observers receive -1 as sentinel
        }
        notifyPassengersRemoved(passengers.size(), availableSeats);
    }
```

```java
    public int getCapacity() throws NoSuchFieldException {
        return this.aircraft.getPassengerCapacity();
    }

    public int getAvailableCapacity() throws NoSuchFieldException {
        return this.getCapacity() - this.passengers.size();
    }

    public Date getDepartureTime() { return departureTime; }
    public List<Passenger> getPassengers() { return passengers; }
    public double getCurrentPrice() { return currentPrice; }

    /** Captures old price, updates, then notifies all observers. */
    public void setCurrentPrice(double currentPrice) {
        double oldPrice = this.currentPrice;
        this.currentPrice = currentPrice;
        notifyPriceChange(oldPrice, currentPrice);
    }
}
```

Key Modifications:

1. `implements FlightSubject` added to class declaration
2. `private final List<FlightObserver> observers` field added; initialised in both constructors
3. `registerObserver()`, `removeObserver()`, `notifyObservers()` implement the subject interface
4. Private helpers `notifyPriceChange()`, `notifyPassengersAdded()`, `notifyPassengersRemoved()` iterate a snapshot of the list
5. `addPassengers()` and `removePassengers()` call their respective notify helpers after mutating state
6. `setCurrentPrice()` — key change — captures old price and calls `notifyPriceChange()` after the update

---

7.4 Concrete Observer: CustomerNotificationObserver

```java
package flight.reservation.flight;

import flight.reservation.Customer;

/** Concrete Observer — sends email-style notifications to a Customer on flight
events. */
public class CustomerNotificationObserver implements FlightObserver {

    private final Customer customer;

    public CustomerNotificationObserver(Customer customer) {
        this.customer = customer;
    }

    @Override
    public void onPriceChanged(ScheduledFlight flight, double oldPrice, double
```

```
newPrice) {
        double diff = newPrice - oldPrice;
        String direction = diff > 0 ? "increased" : "decreased";
        System.out.println("----------------------------------");
        System.out.println("[EMAIL TO: " + customer.getEmail() + "]");
        System.out.println("Dear " + customer.getName() + ",");
        System.out.println();
        System.out.println("Price Update for Flight " + flight.getNumber());
        System.out.println("  From: " + flight.getDeparture().getCode()
                + " -> To: " + flight.getArrival().getCode());
        System.out.printf("  Old Price: $%.2f%n", oldPrice);
        System.out.printf("  New Price: $%.2f%n", newPrice);
        if (diff > 0) {
            System.out.printf("  [!] Price %s by $%.2f%n", direction,
Math.abs(diff));
        } else {
            System.out.printf("  [v] Price %s by $%.2f - Book now to save
money!%n", direction, Math.abs(diff));
        }
        System.out.println("----------------------------------");
        System.out.println();
    }

    @Override
    public void onPassengersAdded(ScheduledFlight flight, int passengersAdded, int
availableSeats) {
        System.out.println("[" + customer.getName() + "] " + passengersAdded
                + " passenger(s) just booked Flight " + flight.getNumber()
                + ". Available seats remaining: " + availableSeats);
    }

    @Override
    public void onPassengersRemoved(ScheduledFlight flight, int passengersRemoved,
int availableSeats) {
        System.out.println("[" + customer.getName() + "] " + passengersRemoved
                + " passenger(s) cancelled on Flight " + flight.getNumber()
                + ". Available seats now: " + availableSeats);
    }

    public Customer getCustomer() {
        return customer;
    }
}
```

7.5 Concrete Observer: PriceAlertObserver

```
package flight.reservation.flight;

import flight.reservation.Customer;
```

```java
/** Concrete Observer — fires an alert when price drops to or below targetPrice.
*/
public class PriceAlertObserver implements FlightObserver {

    private final Customer customer;
    // alert only when newPrice <= targetPrice
    private final double targetPrice;

    public PriceAlertObserver(Customer customer, double targetPrice) {
        this.customer = customer;
        this.targetPrice = targetPrice;
    }

    @Override
    public void onPriceChanged(ScheduledFlight flight, double oldPrice, double
newPrice) {
        if (newPrice <= targetPrice) {
            System.out.println("*** PRICE ALERT ***");
            System.out.println("Customer: " + customer.getName());
            System.out.printf("Flight %d is now $%.2f%n", flight.getNumber(),
newPrice);
            System.out.printf("Your target price was: $%.2f%n", targetPrice);
            System.out.println("[TARGET REACHED! Book now!]");
            System.out.println("--------------------------------");
            System.out.println();
        }
    }

    @Override
    public void onPassengersAdded(ScheduledFlight flight, int passengersAdded, int
availableSeats) {
        // no-op — only watches price changes
    }

    @Override
    public void onPassengersRemoved(ScheduledFlight flight, int passengersRemoved,
int availableSeats) {
        // no-op — only watches price changes
    }

    public double getTargetPrice() {
        return targetPrice;
    }

    public Customer getCustomer() {
        return customer;
    }
}
```

7.6 Concrete Observer: AdminMonitorObserver

```java
package flight.reservation.flight;

/** Concrete Observer — collects analytics for an admin dashboard (price changes,
bookings, cancellations). */
public class AdminMonitorObserver implements FlightObserver {

    private final String adminName;
    private int totalPriceChanges = 0;
    private int totalPassengerBookings = 0;
    private int totalPassengerCancellations = 0;

    public AdminMonitorObserver(String adminName) {
        this.adminName = adminName;
    }

    @Override
    public void onPriceChanged(ScheduledFlight flight, double oldPrice, double
newPrice) {
        totalPriceChanges++;
        double percentChange = oldPrice == 0 ? 0 : ((newPrice - oldPrice) /
oldPrice) * 100;

        double occupancyPercent = 0;
        try {
            int total = flight.getCapacity();
            int booked = total - flight.getAvailableCapacity();
            occupancyPercent = total == 0 ? 0 : ((double) booked / total) * 100;
        } catch (NoSuchFieldException ignored) {
            // Not all aircraft types expose capacity; silently default to 0%
        }

        System.out.println("[ADMIN DASHBOARD - " + adminName + "]");
        System.out.println("==================================");
        System.out.println("Event: Price Change");
        System.out.println("Flight: " + flight.getNumber());
        System.out.printf("Old: $%.2f -> New: $%.2f%n", oldPrice, newPrice);
        System.out.printf("Change: %.1f%%%n", percentChange);
        System.out.println("Total Price Changes Monitored: " + totalPriceChanges);
        System.out.printf("Current Occupancy: %.1f%%%n", occupancyPercent);
        System.out.println("==================================");
        System.out.println();
    }

    @Override
    public void onPassengersAdded(ScheduledFlight flight, int passengersAdded, int
availableSeats) {
        totalPassengerBookings += passengersAdded;
        System.out.println("[ADMIN - " + adminName + "] Passengers booked: "
                + passengersAdded + " on Flight " + flight.getNumber());
        System.out.println("Total bookings tracked: " + totalPassengerBookings);
        System.out.println();
    }
```

```
    @Override
    public void onPassengersRemoved(ScheduledFlight flight, int passengersRemoved,
int availableSeats) {
        totalPassengerCancellations += passengersRemoved;
        System.out.println("[ADMIN - " + adminName + "] Passengers cancelled: "
                + passengersRemoved + " on Flight " + flight.getNumber());
        System.out.println("Total cancellations tracked: " +
totalPassengerCancellations);
        System.out.println();
    }

    public int getTotalPriceChanges() { return totalPriceChanges; }
    public int getTotalPassengerBookings() { return totalPassengerBookings; }
    public int getTotalPassengerCancellations() { return
totalPassengerCancellations; }
    public String getAdminName() { return adminName; }
}
```

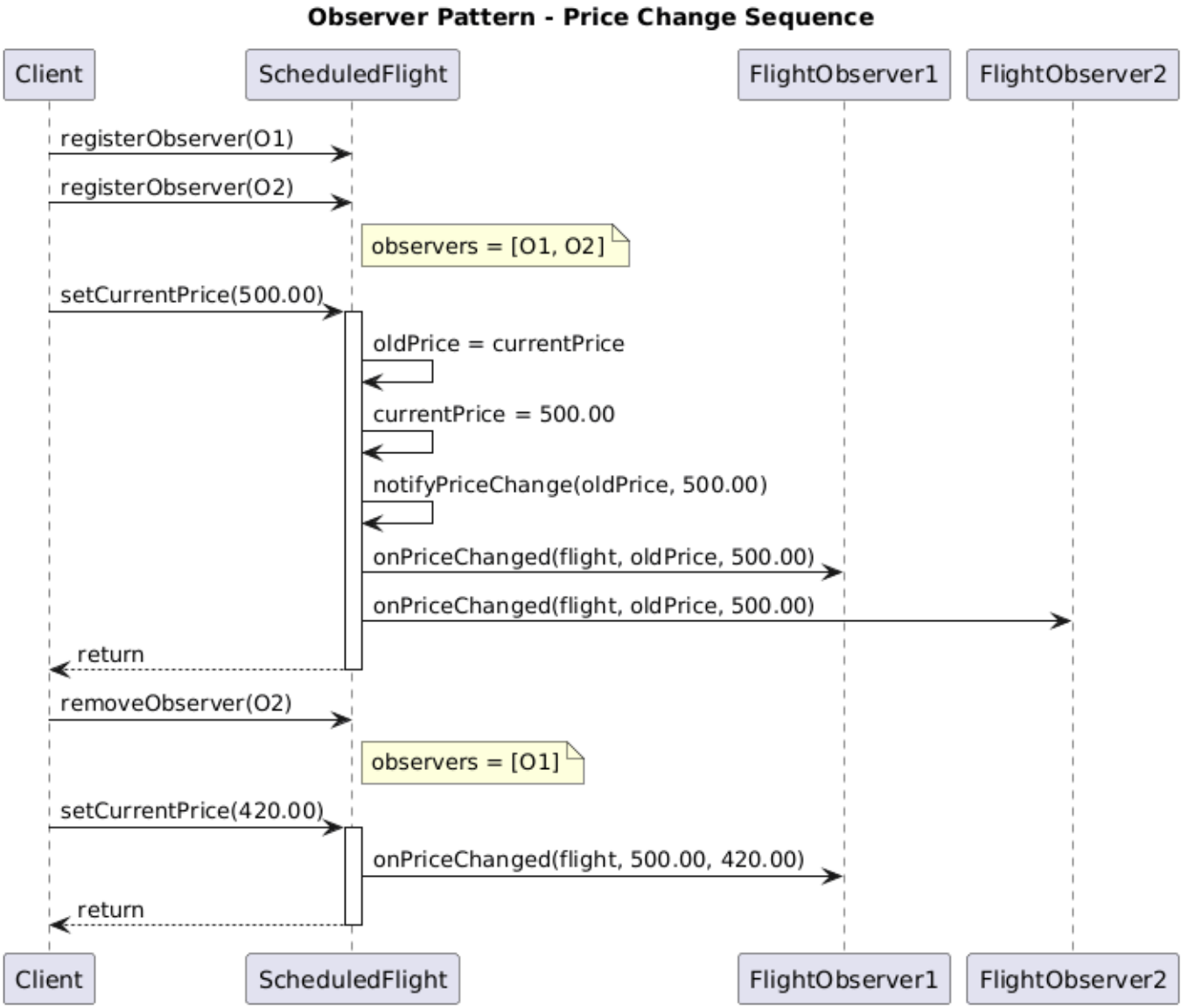
7.7 Usage Example / Demonstration

```
package flight.reservation;

import flight.reservation.flight.AdminMonitorObserver;
import flight.reservation.flight.CustomerNotificationObserver;
import flight.reservation.flight.PriceAlertObserver;
import flight.reservation.flight.ScheduledFlight;
import flight.reservation.plane.AircraftFactory;

import java.time.Instant;
import java.util.Date;

/** Observer Pattern usage demonstration. */
public class ObserverPatternDemo {

    public static void main(String[] args) throws NoSuchFieldException {

        // Setup
        Airport jfk = new Airport("John F. Kennedy International Airport", "JFK",
"Queens, New York, New York");
        Airport lax = new Airport("Los Angeles International Airport", "LAX", "Los
Angeles, California");
        ScheduledFlight flight = new ScheduledFlight(
                123, jfk, lax,
                AircraftFactory.createPassengerPlane("A380"),
                Date.from(Instant.now()),
                500.00
        );

        // Create customers
        Customer alice = new Customer("Alice", "alice@email.com");
```

```
        Customer bob = new Customer("Bob", "bob@email.com");

        // Observers
        CustomerNotificationObserver aliceNotifier = new
CustomerNotificationObserver(alice);
        PriceAlertObserver bobPriceAlert = new PriceAlertObserver(bob, 450.00);
        AdminMonitorObserver adminMonitor = new AdminMonitorObserver("John
Admin");
        System.out.println("========== REGISTERING OBSERVERS ==========");
        System.out.println();
        flight.registerObserver(aliceNotifier);
        flight.registerObserver(bobPriceAlert);
        flight.registerObserver(adminMonitor);

        // Scenario 1: Price Increase
        System.out.println("========== SCENARIO 1: PRICE INCREASE ==========");
        System.out.println();
        flight.setCurrentPrice(550.00);

        // Scenario 2: Price Drops to Bob's Target
        System.out.println("========== SCENARIO 2: PRICE DROPS TO TARGET
==========");
        System.out.println();
        flight.setCurrentPrice(440.00);

        // Scenario 3: Passengers Book
        System.out.println("========== SCENARIO 3: PASSENGERS BOOKING
==========");
        System.out.println();
        flight.addPassengers(java.util.Arrays.asList(new Passenger("P1"), new
Passenger("P2")));

        // Scenario 4: Bob Unsubscribes
        System.out.println("========== SCENARIO 4: BOB UNSUBSCRIBES ==========");
        System.out.println();
        flight.removeObserver(bobPriceAlert);

        // Scenario 5: Price Change After Bob Unsubscribed
        System.out.println("========== SCENARIO 5: PRICE CHANGE (BOB WON'T SEE
THIS) ==========");
        System.out.println();
        flight.setCurrentPrice(420.00);

        System.out.println("========== DEMO COMPLETE ==========");
    }
}
```

---

8. SEQUENCE DIAGRAM



---

9. REAL-WORLD APPLICATIONS

Where Observer Pattern is Used:

1. **Kayak / Google Flights**
   - Price tracking alerts
   - Seat availability notifications
2. **Event Management Systems**
   - Java Swing (ActionListener)
   - Android (OnClickListener)
3. **Stock Market Applications**
   - Stock price updates
   - Portfolio value changes
4. **Social Media**

    - Follower notifications
    - News feed updates

5. **Model-View-Controller (MVC)**
    - Model notifies View of data changes

---

10. SUMMARY

Pattern Applied: Observer Pattern

Location: `ScheduledFlight.setCurrentPrice()` (line 82)

Problem Solved: Silent price/capacity changes -> Active notifications

Impact:

- **Modularity**: Easy to add new notification types (SMS, push, email)
- **Scalability**: Each flight can have unlimited observers
- **Maintainability**: Notification logic separate from flight logic
- **Business Value**: Enables modern booking features (price alerts, notifications)
- **Code Quality**: Follows SOLID principles (SRP, OCP, DIP)

Why this should be done:

1. Real airline apps use this exact pattern (Kayak, Expedia, Google Flights)
2. Clear Subject-Observer relationship (Flight -> Customers)
3. Immediate business value (customer satisfaction, increased sales)
4. Natural fit - price changes ARE events that need broadcasting
5. Multiple stakeholders care about the same event

---

11. CONCLUSION

The Observer Pattern is the most appropriate and evident design pattern for the `ScheduledFlight` price change functionality in this reservation system. It transforms a simple setter method into a powerful notification system that:

- **Decouples** flight management from notification logic
- **Enables** real-time price tracking and alerts
- **Supports** unlimited observer types without code changes
- **Follows** industry best practices (used by all major booking platforms)
- **Improves** user experience through proactive notifications

By implementing this pattern, the system gains flexibility, extensibility, and real business value while maintaining clean, maintainable code architecture.

END OF DETAILED OBSERVER PATTERN ANALYSIS